

Delivery 3: Security Hardening (DevSecOps)

Security Hardening Report

Project: Vulnerable Node (cr0hn/vulnerable-node)

Date: 2026-03-07

Authors: Team 17001969 / 17001625

Table of Contents

1. [Overview](#)
 2. [Architecture Compliance](#)
 3. [Task 1: SBOM Generation](#)
 4. [Task 2: Vulnerability Scanning and Patching](#)
 5. [Task 3: Secret Protection \(Pre-Commit Hook\)](#)
 6. [CI/CD Security Integration](#)
 7. [Developer Guide: Running Security Tools](#)
 8. [Artifacts Summary](#)
-

1. Overview

This delivery implements **Supply Chain Security Hardening** for the Vulnerable Node application, building upon the findings from Delivery 1 (Discovery & Reverse Engineering) and Delivery 2 (Governance & Technical Debt Audit).

The security hardening focuses on three pillars:

1. **Transparency** — Generating a Software Bill of Materials (SBOM) to enumerate all dependencies.

2. **Vulnerability Management** — Scanning for and patching dependency and code-level vulnerabilities.

3. **Prevention** — Installing pre-commit hooks to block accidental secret leaks.

All changes adhere to the **Monolithic but Modular Architecture** constraint. Security fixes are localized within their respective modules (`model/auth.js` , `model/products.js`), maintaining loose coupling and clear module boundaries. No cross-module coupling was introduced.

2. Architecture Compliance

Modular Monolith Preservation

All security improvements were implemented respecting the existing module boundaries:

<code>model/auth.js</code>	Secure refactoring	Parameterized SQL queries (localized to IAM module)
<code>model/products.js</code>	Secure refactoring	Parameterized SQL queries (localized to Catalog/Sales module)
<code>package.json</code>	Dependency update	<code>minimatch</code> vulnerability patched via <code>npm audit fix</code>
<code>.husky/pre-commit</code>	New infrastructure	Standalone pre-commit hook (no module coupling)
<code>security/</code>	New artifacts	SBOM and vulnerability reports (passive, no logic coupling)
<code>.github/workflows/</code>	CI extension	Parallel security pipeline (does not modify existing governance pipeline)

No microservices were introduced. No tight coupling was created between modules. Each security fix remains self-contained within its respective bounded context, as described in the Delivery 1 Context Map.

3. Task 1: SBOM Generation

What is an SBOM?

A Software Bill of Materials (SBOM) is a structured inventory of all software components (direct and transitive dependencies) used in the project. It enables supply chain transparency, license compliance auditing, and vulnerability tracking.

Implementation

The SBOM is generated using **Trivy v0.69.3** in **CycloneDX JSON format**, which is an industry-standard format supported by OWASP and recognized by regulatory frameworks.

Command used:

```
trivy fs --format cyclonedx -o security/sbom.json .
```

Output location: `security/sbom.json`

SBOM Contents

The generated SBOM catalogs all production dependencies of the project including:

- **Direct dependencies:** `express`, `pg-promise`, `ejs`, `express-session`, `body-parser`, `cookie-parser`, `log4js`, `morgan`, `serve-favicon`, `express-ejs-layouts`, `debug`
- **Transitive dependencies:** All nested sub-dependencies resolved from `package-lock.json`
- **Metadata:** Package versions, PURL identifiers, dependency relationships, license information

Regeneration

To regenerate the SBOM (e.g., after adding new dependencies):

```
npm run security:sbom
```

This script is defined in `package.json` and executes the Trivy SBOM generation command.

Automation

SBOM generation is automated in the CI/CD pipeline via [.github/workflows/security-scan.yml](#). Every push and pull request triggers automatic SBOM generation. The SBOM is uploaded as a build artifact with 30-day retention.

4. Task 2: Vulnerability Scanning and Patching

4.1 Pre-Patch Vulnerability Scan

A Trivy vulnerability scan was performed **before** applying any fixes. The scan identified **3 HIGH severity vulnerabilities** in the [minimatch](#) dependency:

CVE-2026-26996	HIGH	minimatch	5.1.6	5.1.7+	Denial of Service via specially crafted glob patterns
CVE-2026-27903	HIGH	minimatch	5.1.6	5.1.8+	Denial of Service due to unbounded recursive backtracking via crafted glob patterns
CVE-2026-27904	HIGH	minimatch	5.1.6	5.1.8+	Denial of Service via catastrophic backtracking in glob expressions

All three vulnerabilities are **ReDoS (Regular Expression Denial of Service)** attacks that could cause the application or its build tools to hang when processing malicious glob patterns.

Full pre-patch report: [security/reports/vuln-before.json](#)

4.2 Code-Level Security Hotspots

In addition to the dependency vulnerability, two critical code-level security hotspots were identified in Delivery 2's Technical Debt Audit:

1. **SQL Injection in** `model/auth.js` (CWE-89) — Authentication bypass via string concatenation
2. **SQL Injection in** `model/products.js` (CWE-89) — Data exfiltration and manipulation via string concatenation in 4 functions

These were flagged in `scanner/sast/expectedIssues.csv` as CWE-89 vulnerabilities at lines 9 (auth.js), 16, 23, and 40 (products.js).

4.3 Patches Applied

Patch 1: Dependency Update (minimatch)

Action: `npm audit fix`

Result: All 3 HIGH severity `minimatch` CVEs were resolved by updating to a patched version. The `package-lock.json` was updated automatically.

Patch 2: SQL Injection Fix in `model/auth.js` (IAM Module)

Vulnerability: CWE-89 — SQL Injection allowing authentication bypass.

Before (vulnerable):

```
function do_auth(username, password) {  
  var db = pgp(config.db.connectionString);  
  var q = "SELECT * FROM users WHERE name = '" + username + "' AND password = '" + password + "  
  return db.one(q);  
}
```

An attacker could inject `admin' OR '1'='1` as the username to bypass authentication entirely.

After (secure):

```
function do_auth(username, password) {  
  var db = pgp(config.db.connectionString);  
  var q = "SELECT * FROM users WHERE name = $1 AND password = $2;";
```

```

    return db.one(q, [username, password]);
  }

```

The fix uses **pg-promise parameterized queries** (`$1`, `$2` placeholders). User input is never concatenated into the SQL string. Parameters are passed as a separate array, preventing any SQL injection.

Scope: Localized to `model/auth.js` (IAM bounded context). No other modules were affected.

Patch 3: SQL Injection Fix in `model/products.js` (Catalog/Sales Module)

Vulnerability: CWE-89 — SQL Injection in 4 functions: `getProduct()`, `search()`, `purchase()`, `get_purchased()`.

Before (vulnerable — example from `search()`):

```

function search(query) {
  var q = "SELECT * FROM products WHERE name ILIKE '%" + query + "%' OR description ILIKE '%" + query + "%'";
  return db.many(q);
}

```

After (secure):

```

function search(query) {
  var q = "SELECT * FROM products WHERE name ILIKE $1 OR description ILIKE $1;";
  return db.many(q, ['%' + query + '%']);
}

```

All four functions in `model/products.js` were refactored to use parameterized queries:

Function	Placeholder(s)	Description
<code>getProduct(product_id)</code>	<code>\$1</code> = product_id	Product lookup by ID
<code>search(query)</code>	<code>\$1</code> = <code>'%' + query + '%'</code>	Product search with ILIKE
<code>purchase(cart)</code>	<code>\$1</code> – <code>\$8</code> = cart fields	Purchase insertion
<code>get_purchased(username)</code>	<code>\$1</code> = username	Purchase history lookup

Scope: Localized to `model/products.js` (Catalog/Sales bounded context). The `routes/products.js` handler was not modified — the module interface (function signatures and return types) was preserved.

4.4 Post-Patch Vulnerability Scan

After applying all patches, a second Trivy scan confirmed **0 vulnerabilities**:

Report Summary

Target	Type	Vulnerabilities
package-lock.json	npm	0

Full post-patch report: `security/reports/vuln-after.json`

4.5 Test Suite Verification

All 55 tests in 7 test suites pass after the patches. The test assertions were updated to validate the new parameterized query pattern (e.g., checking for `$1`, `$2` placeholders and separate parameter arrays instead of inline values).

Test Suites: 7 passed, 7 total
Tests: 55 passed, 55 total
Statement coverage: 98.38%

4.6 Before vs. After Summary

Dependency vulnerabilities (Trivy)	3 HIGH	0
SQL injection points (CWE-89)	5 locations	0
Test suite	All passing	All passing
Statement coverage	98.38%	98.38%

5. Task 3: Secret Protection (Pre-Commit Hook)

Purpose

Prevent accidental commits of hardcoded secrets (API keys, tokens, passwords, private keys) into the repository. This is a critical supply chain security control that catches secrets before they enter version control history.

Implementation

Tool: Husky v9 (Git hooks manager for Node.js projects)

Installation:

```
npm install --save-dev husky
npx husky init
```

Hook location: `.husky/pre-commit`

How It Works

The pre-commit hook executes automatically before every `git commit`. It:

1. Scans **only staged changes** (`git diff --cached`) — not the entire codebase
2. Searches for secret patterns using case-insensitive regex matching
3. **Blocks the commit** if any secret pattern is detected (exit code 1)
4. Displays a clear error message identifying the detected patterns

Secret Patterns Detected

<code>API_KEY=</code>	<code>API_KEY=sk-abc123 ...</code>
<code>SECRET=</code>	<code>SECRET=my-secret-value</code>
<code>TOKEN=</code>	<code>TOKEN=ghp_xxxx ...</code>
<code>PASSWORD=</code>	<code>PASSWORD=admin123</code>
<code>PRIVATE_KEY=</code>	<code>PRIVATE_KEY=-----BEGIN RSA ...</code>

AWS_ACCESS_KEY	AWS_ACCESS_KEY=AKIAIOSFODNN ...
AWS_SECRET_KEY	AWS_SECRET_KEY=wJalrXUtnFEMI ...
DB_PASSWORD=	DB_PASSWORD=postgres
SESSION_SECRET=	SESSION_SECRET=mysecret ...

Example Behavior

When a secret is detected:

```
Running pre-commit secret detection scan...
```

```
=====
ERROR: Secret detected in staged files. Commit blocked.
=====
```

```
The following secret patterns were found in your staged changes:
```

```
+API_KEY=sk-1234567890abcdef
```

```
Please remove any hardcoded secrets before committing.
Use environment variables or a .env file instead.
```

When no secrets are found:

```
Running pre-commit secret detection scan...
```

```
No secrets detected. Proceeding with commit.
```

Architecture Impact

The pre-commit hook is a **standalone infrastructure component** in the `.husky/` directory. It:

- Does not modify any application module
- Does not introduce dependencies between bounded contexts
- Runs in the developer's local Git environment only
- Has zero impact on application runtime behavior

6. CI/CD Security Integration

Existing Pipeline

The repository already has a Governance CI pipeline (`.github/workflows/governance.yml`) that runs ESLint complexity checks and Jest coverage tests.

New Security Pipeline

A new **parallel** CI workflow was added at `.github/workflows/security-scan.yml` . It runs alongside (not replacing) the governance pipeline.

Triggers: Every push to any branch and every pull request.

Pipeline Steps:

1	Checkout code	Get source code
2	Setup Node.js 22	Runtime environment
3	<code>npm ci</code>	Install locked dependencies
4	Install Trivy	Security scanner
5	Generate SBOM	<code>trivy fs --format cyclonedx -o security/sbom.json .</code>
6	Vulnerability scan	<code>trivy fs --scanners vuln --format json</code> + table
7	Security gate	<code>trivy fs --exit-code 1 --severity HIGH,CRITICAL</code> — fails build if HIGH/CRITICAL vulns found
8	Upload SBOM artifact	30-day retention
9	Upload vuln report artifact	30-day retention

Security Gate

Step 7 acts as a **quality gate**: if Trivy finds any HIGH or CRITICAL vulnerability in the dependency tree, the build fails with exit code 1. This prevents merging pull requests that introduce vulnerable dependencies.

Integration with Existing Pipeline

The security pipeline is fully independent from the governance pipeline: - Different workflow file, different job name - No shared state or artifacts between pipelines - Both pipelines run in parallel on the same triggers - Failure in one pipeline does not affect the other

7. Developer Guide: Running Security Tools

Prerequisites

- **Node.js 22+** installed
- **Trivy** installed ([installation guide](#))
- **Git** configured (for Husky hooks)

After Cloning

```
npm install          # Installs dependencies + sets up Husky hooks automatically
```

The `prepare` script in `package.json` runs `husky` on `npm install`, configuring Git hooks automatically.

Available Security Commands

<code>npm run security:sbom</code>	Generate SBOM in CycloneDX format → <code>security/sbom.json</code>
<code>npm run security:scan</code>	Run Trivy vulnerability scan (table output to console)

<code>npm run security:scan:json</code>	Run Trivy scan with JSON report → <code>security/reports/vuln-latest.json</code>
<code>npm audit</code>	Run npm's built-in audit (alternative to Trivy for dependency-only checks)

Regenerating Reports

SBOM:

```
npm run security:sbom
```

Vulnerability scan:

```
npm run security:scan          # Human-readable table
npm run security:scan:json     # Machine-readable JSON
```

Bypassing the Pre-Commit Hook

In exceptional cases (e.g., committing a test fixture that intentionally contains a pattern), you can bypass the hook:

```
git commit --no-verify -m "your message"
```

Warning: This should only be used in justified situations. All bypasses should be documented in the commit message.

8. Artifacts Summary

SBOM	<code>security/sbom.json</code>	CycloneDX JSON Software Bill of Materials

Pre-patch vulnerability report	<code>security/reports/vuln-before.json</code>	Trivy scan showing 3 HIGH vulnerabilities
Post-patch vulnerability report	<code>security/reports/vuln-after.json</code>	Trivy scan showing 0 vulnerabilities
Pre-commit hook	<code>.husky/pre-commit</code>	Secret detection hook blocking sensitive patterns
Security CI workflow	<code>.github/workflows/security-scan.yml</code>	Automated Trivy scanning and SBOM generation
This documentation	Delivery_3_Security_Hardening_(DevSecOps)/Security_Hardening_Report.md	Technical report

Files Modified

<code>model/auth.js</code>	SQL injection fix — parameterized queries
<code>model/products.js</code>	SQL injection fix — parameterized queries (4 functions)
<code>package.json</code>	Added Husky, security scripts
<code>package-lock.json</code>	minimatch updated, Husky added
<code>__tests__/auth.test.js</code>	Updated assertions for parameterized queries
<code>__tests__/products-model.test.js</code>	Updated assertions for parameterized queries

Files Created

<code>security/sbom.json</code>	SBOM artifact
<code>security/reports/vuln-before.json</code>	Pre-patch vulnerability report
<code>security/reports/vuln-after.json</code>	Post-patch vulnerability report
<code>.husky/pre-commit</code>	Secret detection pre-commit hook
<code>.github/workflows/security-scan.yml</code>	Security CI pipeline
<code>Delivery_3_Security_Hardening_(DevSecOps)/Security_Hardening_Report.md</code>	This document